



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Реализация стека и дека

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Структура стека	5
2.1.1 Добавление элемента	5
2.1.2 Удаление элемента	6
2.1.3 Проверка стека на пустоту	6
2.1.4 Количество элементов	6
2.1.5 Вывод данных	6
2.2 Структура дека	6
2.2.1 Добавление элемента	7
2.2.2 Удаление элемнента	7
3 Технологическая часть	8
4 Исследовательская часть	16
Заключение	18
Список используемых источников	19

Введение

Цель лабораторной работы: разработать стек и дек.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать две схемы хранения данных в стеке и деке.
2. Разработать алгоритмы, необходимые для хранения данных.
3. Реализовать разработанные алгоритмы.
4. Выполнить тестирование реализации разработанных алгоритмов.

Согласно варианту, требуется разработать следующие две схемы.

1. Стек. Написать функции добавления элементов, удаления элементов и вывод содержимого в консоль.
2. Дек. разработать функции добавления, удаления данных и их вывод.

1 Аналитическая часть

Стек[1] — абстрактный тип данных, представляющий собой список элементов, организованных по принципу LIFO (англ. last in — first out, «последним пришёл — первым вышел»). Стек является очередью с одной стороной входа и выхода элементов.

Дек[1] — структура данных типа «список», функционирующая одновременно по двум принципам организации данных: LIFO и FIFO (англ. first in — first out, «первым пришёл — первым вышел»). Определить дек можно как очередь с двумя сторонами, так и стек, имеющий два конца. То есть данный подвид списка характерен двухсторонним доступом: выполнение поэлементной операции, определенной над деком, предполагает возможность выбора одной из его сторон в качестве активной.

2 Конструкторская часть

2.1 Структура стека

Стек состоит из ячеек, которые представлены в виде структуры, содержащей какие-либо данные и указатель на следующий элемент. Для удобного восприятия стек представлен на рисунке 1.



Рис. 1 – Схематичное представление стека

Как можно увидеть стек хранит в одном блоке данные и “*next” — ссылку на последующий элемент. Иными словами в ячейке “*next” хранится адрес на следующую ячейку.

Над стеком и его элементами можно выполнять следующие операции:

- Добавление элемента в стек.
- Удаление элемента из стека.
- Проверка стека на пустоту.
- Определение количества элементов в стеке.
- Вывод всего стека.

2.1.1 Добавление элемента

Если в стеке еще не было элементов, то создается блок, в котором в ячейку “данные” заносится необходимая информация для хранения. Поскольку после этого блока нет больше следующих блоков, то ячейка “*next” имеет нулевой указатель. При добавлении блока,

если уже были созданы блоки, создается новый блок и ячейка “*next” в предыдущем блоке начинает ссылаться на созданный блок, а в ячейку “данные” добавляется информация.

2.1.2 Удаление элемента

При удалении данных, предпоследний блок изменяет ячейку “*next” на нулевой указатель, а данные из последнего блока удаляются.

2.1.3 Проверка стека на пустоту

Чтобы проверить стек на наличие элементов, необходимо посчитать количество блоков в стеке. Если количество будет больше единицы, то стек не пустой.

2.1.4 Количество элементов

Для подсчета количество элементов в стеке нужно создать курсор который будет идти по ячейкам “*next” и считать их количество, пока не встретится нулевой указатель.

2.1.5 Вывод данных

Аналогично подсчету элементов необходимо создать курсор, который будет идти по ссылкам каждого блока и выводить данные из блока стека.

2.2 Структура дека

Аналогично стеку содержит блоки в которых хранятся данные и указатель на следующий элемент. Дек схематично изображен на рисунке 2.

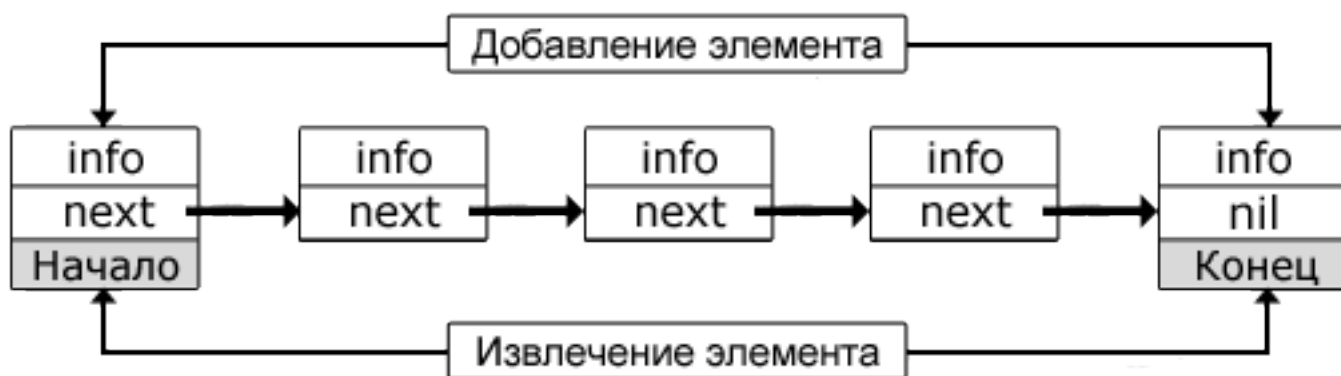


Рис. 2 – Схематичное представление дека

Дек имеет аналогичную структуру, как и стек, в отличие от того, что добавлять и удалять элементы можно с двух сторон (с конца и с начала). Все остальные функции работают аналогично, как и в стеке.

2.2.1 Добавление элемента

При добавлении элемента в дек необходимо написать две функции, которые добавляют элемент либо в начало либо в конец структуры. Если есть необходимость добавить блок в начало то, создается блок и в ячейку “next” добавляется ссылка на бывший начальный элемент, а в ячейку “info” заносятся необходимые данные. При добавлении блока в конец дека ячейка “next” последнего элемента изменяется на указатель на вновь созданный блок. Данные добавляются в ячейку “info”.

2.2.2 Удаление элемента

Как и в добавлении элемента в деке можно удалять элемент с любого конца. При удалении элемента из начала дека ячейки “info” и “next” удаляются методом delete. Если удаляется блок из конца дека, то в предпоследнем блоке ячейку “next” необходимо изменить на нулевую ссылку, а последний блок также удалить, воспользовавшись методом delete.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, была подключена библиотека `<fstream>`. Она включает в себя набор классов, методов и функций, которые предоставляют интерфейс для чтения/записи данных из/в файл. С помощью шаблонных классов “List”, который описывает саму структуру стека и “Node”, который описывает блок стека, был реализован стек на ссылках.

Методы, разработанные в классе, следующие.

- Метод “push_back” — метод, который добавляет элемент в конец стека.
- метод “pop_back” — метод, который удаляет последний элемент из стека.
- Метод “GetSize” — метод, который печатает количество элементов в стеке.
- Метод “check” — метод, который проверяет стек на пустоту.
- Метод “print” — метод, который выводит весь стек в консоль.

В программе данные считываются из файла и заносятся в стек. Далее выводится количество элементов, которое хранится в стеке, удаляется последний элемент и проверяется стек на наличие элементов. Если стек не пустой, то выводятся данные.

Далее приведен листинг реализации стека (листинг 1).

Листинг 1 – Исходный код программы для реализации стека

```
#include <iostream>
#include <fstream>
#include <string>

using namespace std;

template<typename T>
class List {
public:
    List() {
        size = 0;
        head = nullptr;
    }

    ~List() {}

    void push_back(T date) {
        if (head == nullptr) {
            head = new Node<T>(date);
```

```

    }
    else {
        Node<T> *current = this->head;
        while (current->pNext != nullptr) {
            current = current->pNext;
        }
        current->pNext = new Node<T>(date);
    }
    size++;
}

int GetSize() {
    return size;
}

void pop_back() {
    Node<T> *previous = this->head;
    for (int i = 0; i < GetSize() - 2; i++){
        previous = previous->pNext;
    }
    Node<T> *toDelete = previous->pNext;
    previous->pNext = toDelete->pNext;
    delete toDelete;
    size--;
}

void check() {
    if (size == 0) {
        cout << "Стек пустой" << endl;
    }
    else {
        cout << "В стеке хранятся данные " << endl;
    }
}

void print() {
    Node<T> *current = this->head;
    while (current != nullptr) {
        cout << current->date;
        current = current->pNext;
    }
}

```

```
private:
```

```
template<typename T>
class Node {
public:
    Node * pNext;
    T date;

    Node(T date, Node * pNext = nullptr) {
        this->date = date;
        this->pNext = pNext;
    }
};

Node<T> *head;
int size;

};

int main() {
    setlocale(0, "");

    string line;
    ifstream myfile("C:\\Users\\Kononova\\Desktop\\stack_data.txt");

    if (myfile.is_open())
    {
        char c;
        while (myfile.get(c))
        {
            line = line + c;
        }
        myfile.close();
    }
    else
    {
        cout << "Не удалось открыть файл для считывания ";
    }

    List<char> lst;

    for (int i = 0; i < line.size(); i++) {
```

```

        lst.push_back(line[i]);
    }
    cout << "Количество элементов, включая пробелы : " << lst.GetSize() <<
        endl;
    cout << "Удалили последний элемент " << endl;
    lst.pop_back();
    cout << "Добавим в конец 5 " << endl;
    lst.push_back('5');
    lst.check();
    if (lst.GetSize() != 0) {
        lst.print();
    }
    else{
        cout << "стек пустой";
    }

    return 0;
}

```

Также, как и в стеке, дек был реализован с помощью шаблонных классов “List” и “Node”. Методы, характерные для дека, следующие.

- Метод “push_back” — метод, который добавляет элемент в конец дека.
- Метод “push_front” — метод, который добавляет элемент в начало дека.
- метод “pop_back” — метод, который удаляет последний элемент из дека.
- метод “pop_front” — метод, который удаляет первый элемент из дека.
- Метод “GetSize” — метод, который печатает количество элементов в дек.
- Метод “check” — метод, который проверяет дек на пустоту.
- Метод “print” — метод, который выводит весь дек в консоль.

Далее приведен листинг реализации стека (листинг 2).

Листинг 2 – Исходный код программы для реализации дека

```

#include <iostream>
#include <fstream>
#include <string>

using namespace std;

template<typename T>

```

```

class List {
public:
    List() {
        size = 0;
        head = nullptr;
    }

    ~List() {}

    void push_back(T date) {
        if (head == nullptr) {
            head = new Node<T>(date);
        }
        else {
            Node<T> *current = this->head;
            while (current->pNext != nullptr) {
                current = current->pNext;
            }
            current->pNext = new Node<T>(date);
        }
        size++;
    }

    void push_front(T data) {
        head = new Node<T>(data, head);
        size++;
    }

    int GetSize() {
        return size;
    }

    void pop_back() {
        Node<T> *previous = this->head;
        for (int i = 0; i < GetSize() - 2; i++){
            previous = previous->pNext;
        }
        Node<T> *toDelete = previous->pNext;
        previous->pNext = toDelete->pNext;
        delete toDelete;
        size--;
    }
}

```

```

void pop_front(){
    Node<T> *temp = head;
    head = head->pNext;
    delete temp;
    size--;
}

void check() {
    if (size == 0) {
        cout << "Стек пустой" << endl;
    }
    else {
        cout << "В стеке хранятся данные " << endl;
    }
}

void print() {
    Node<T> *current = this->head;
    while (current != nullptr) {
        cout << current->date;
        current = current->pNext;
    }
}

private:

template<typename T>
class Node {
public:
    Node * pNext;
    T date;

    Node(T date, Node * pNext = nullptr) {
        this->date = date;
        this->pNext = pNext;
    }
};

Node<T> *head;
int size;
};

```

```

int main() {
    setlocale(0, "");

    string line;
    ifstream myfile("C:\\Users\\Kononova\\Desktop\\stack_data.txt");

    if (myfile.is_open())
    {
        char c;
        while (myfile.get(c))
        {
            line = line + c;
        }
        myfile.close();
    }
    else
    {
        cout << "Не удалось открыть файл для считывания ";
    }

    List<char> lst;

    for (int i = 0; i < line.size(); i++) {
        lst.push_back(line[i]);
    }
    cout << "Количество элементов, включая пробелы : " << lst.GetSize() <<
        endl;
    cout << "Удалили последний элемент " << endl;
    lst.pop_back();
    cout << "Удалили первый элемент " << endl;
    lst.pop_front();
    cout << "Добавим в начало r " << endl;
    lst.push_front('r');
    cout << "Добавим в конец e " << endl;
    lst.push_back('e');
    lst.check();
    if (lst.GetSize() != 0) {
        lst.print();
    }
    else{
        cout << "Дек пустой";
    }
}

```

```
    }  
  
    return 0;  
}
```

В программе данные считываются из файла и заносятся в дек. Выводится количество элементов, которое хранится в деке, удаляется последний и начальный элемент, добавляется в начало и в конец по элементу и выводятся данные.

4 Исследовательская часть

Далее на рисунках предствалены результаты программы и входной файл, в котором лежат данные.

Результат работы программы для стека приведен на рис. 3.

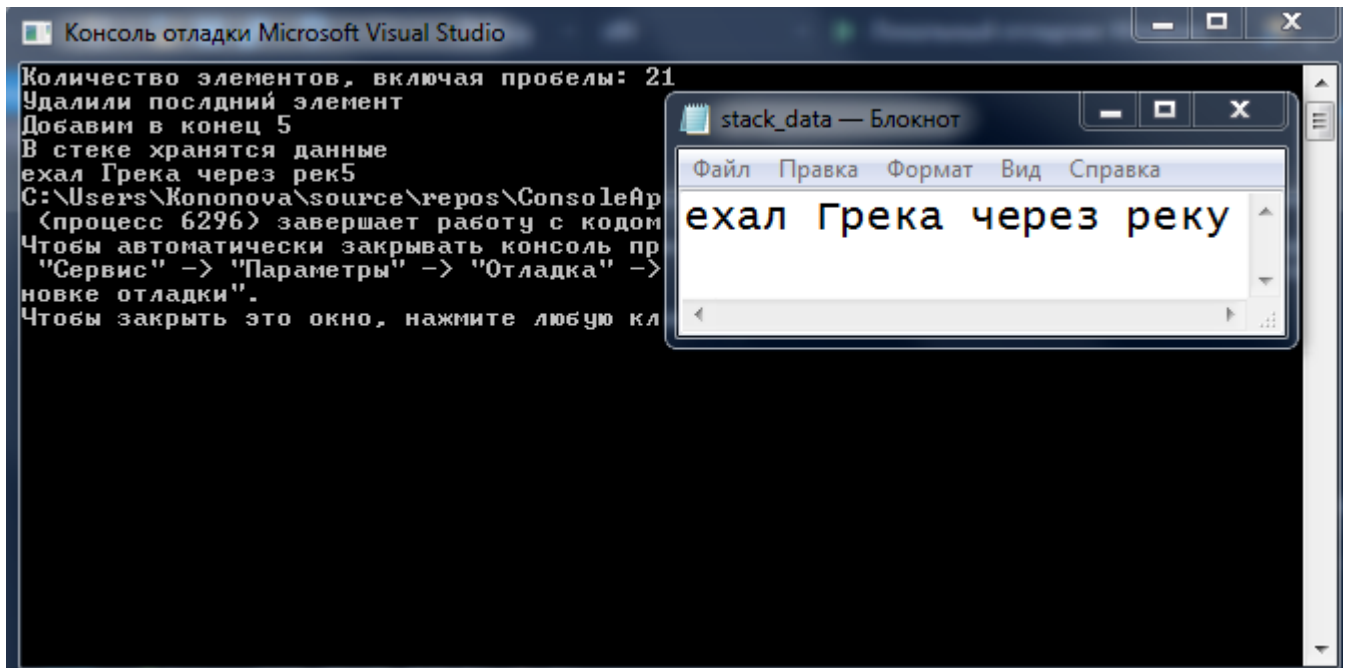


Рис. 3 – Работоспособность стека

Результат работы программы для стека приведен на рис. 4.

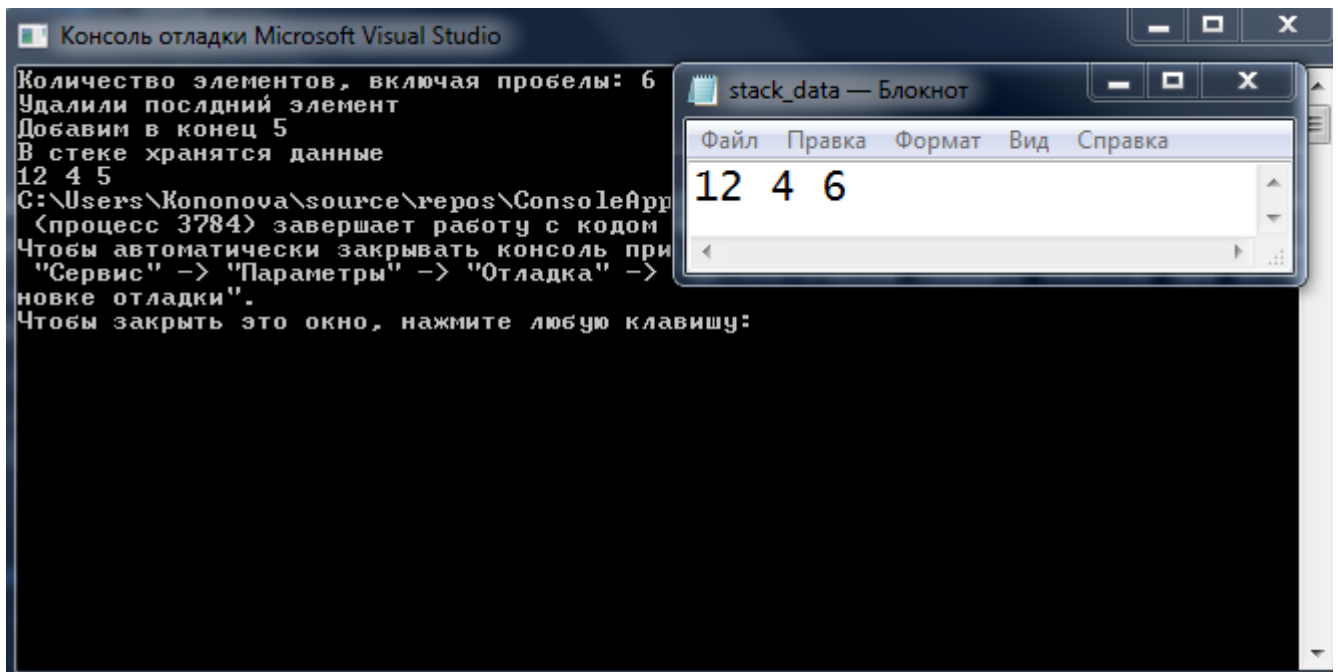


Рис. 4 – Работоспособность стека

Результат работы программы для дека приведен на рис. 5.

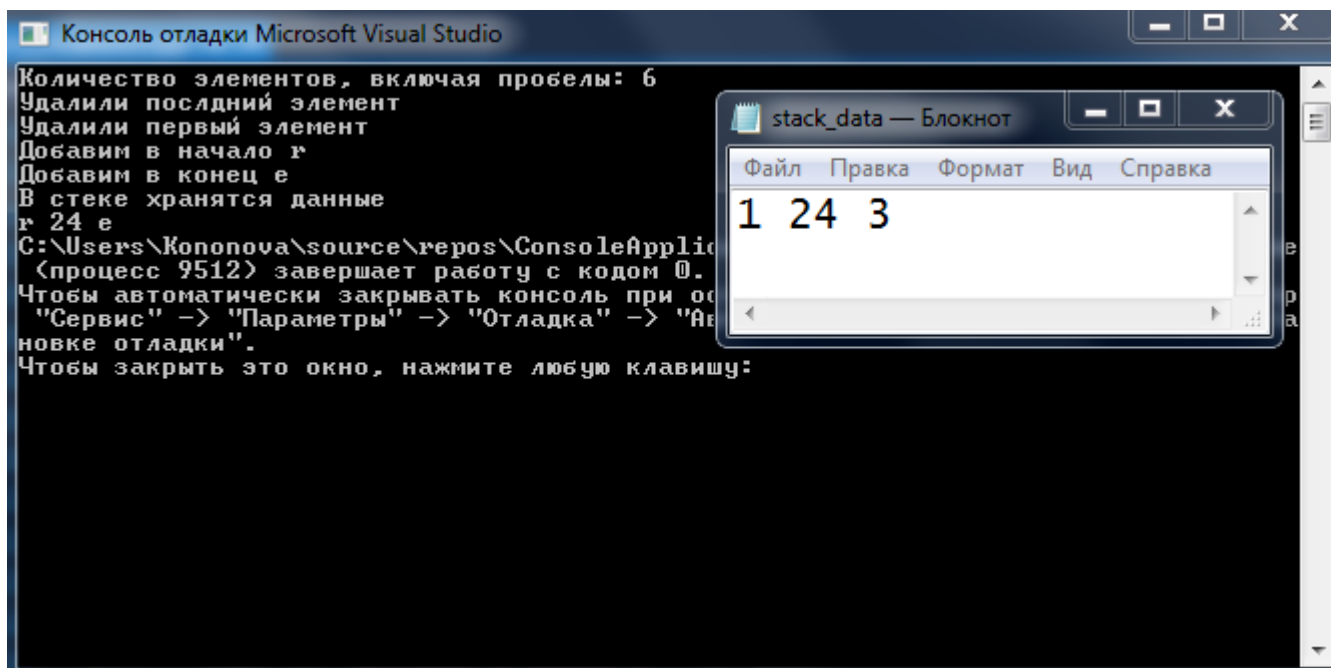


Рис. 5 – Работоспособность дека

Результат работы программы для дека приведен на рис. 6.

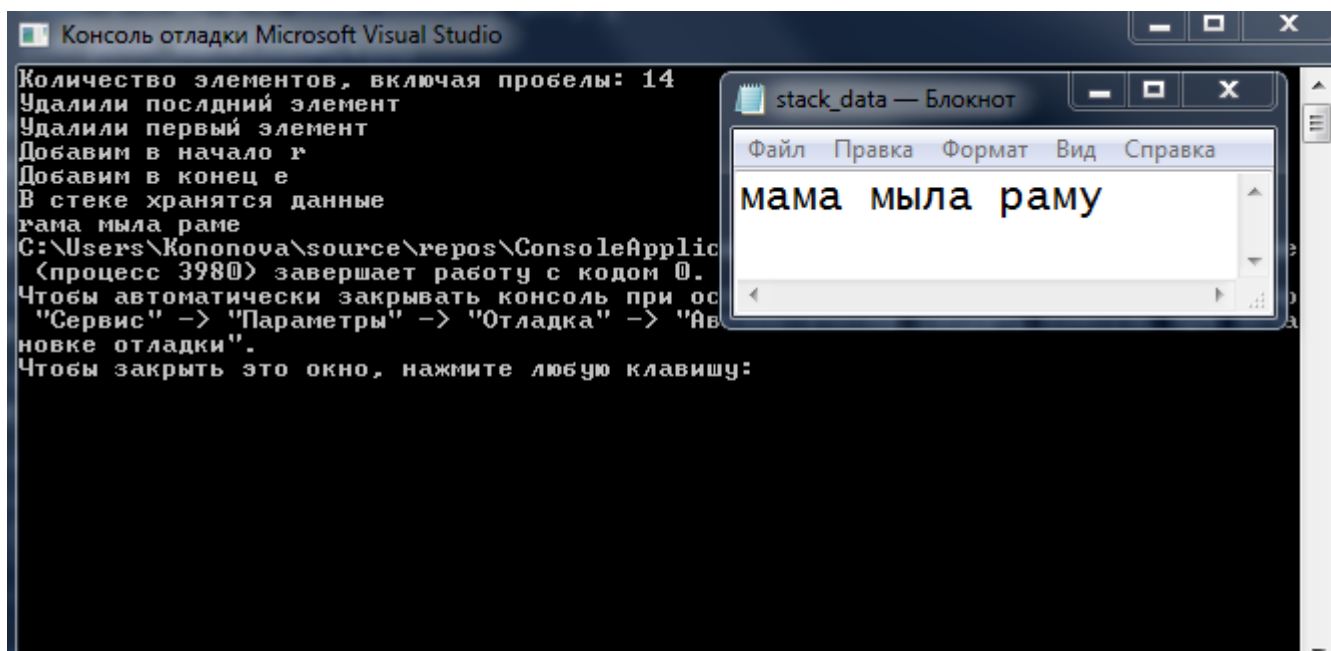


Рис. 6 – Работоспособность дека

Заключение

В результате выполнения лабораторной работы были выполнены все задачи.

1. Описаны две схемы хранения данных в стеке и деке.
2. Разработаны алгоритмы, необходимые для хранения данных.
3. Реализованы разработанные алгоритмы.
4. Выполнено тестирование реализации разработанных алгоритмов.

Согласно варианту, требуется разработаны следующие две схемы.

1. Стек. Написаны функции добавления элементов, удаления элементов и вывод содержимого в консоль.
2. Дек. разработаны функции добавления, удаления данных и их вывод.

Стек и дек необходимы для хранения информации. Дек, в отличие от стека является более гибким хранилищем информации, так как он располагает большим набором функций.

Список литературы

1. Е.А. Конова , Г.А. Поллак, А.М. Ткачев. Структуры данных. Программирование на языке С и С++. Учебное пособие, под ред. Поллак Г.А., Челябинск, Издательство ЮУрГУ, 2004